

HOW ATLAS ENUMERATES THE WEYL GROUP

ABSTRACT. We explain the data structure and algorithm used by the Atlas of Lie Groups and Representations software to represent and enumerate elements of a Weyl group. The central idea, due to Fokko du Cloux, is to factor each group element as a product of minimal-length coset representatives for a chain of parabolic subquotients, encoding the element as a fixed-length array of small integers. Multiplication is implemented by a collection of finite-state transducer automata, one per parabolic subquotient, that were precomputed at group construction time. The reference for the underlying mathematics is du Cloux's 1999 paper [1].

CONTENTS

1. WEYL GROUPS AND COXETER SYSTEMS

A *Weyl group* W is a finite group generated by reflections in a Euclidean space; it is in particular a *Coxeter group*. We fix a set of *simple reflections* (generators) $S = \{s_1, \dots, s_n\}$, where n is the rank. Every element of W can be written as a word in these generators, and the *length* $\ell(w)$ of $w \in W$ is the minimum length of such a word.

The generators satisfy the *Coxeter relations*:

$$s_i^2 = e, \quad (s_i s_j)^{m_{ij}} = e,$$

where $m_{ij} \in \{2, 3, 4, 6\}$ for a Weyl group (the value $m_{ij} = 2$ means s_i and s_j commute). The Coxeter matrix (m_{ij}) encodes the same information as the Cartan matrix and is determined by the Dynkin diagram.

The cardinalities of the Weyl groups of the simple Lie algebras are:

$$|W(A_n)| = (n+1)!, \quad |W(B_n)| = |W(C_n)| = 2^n n!, \quad |W(D_n)| = 2^{n-1} n!,$$

$$|W(E_6)| = 51840, \quad |W(E_7)| = 2903040, \quad |W(E_8)| = 696729600,$$

$$|W(F_4)| = 1152, \quad |W(G_2)| = 12.$$

These grow rapidly; for example $|W(E_8)| \approx 7 \times 10^8$ and $|W(B_{20})| = 2^{20} \cdot 20! \approx 2.7 \times 10^{24}$. Storing all elements as explicit reduced words or permutation matrices is infeasible.

2. PARABOLIC SUBGROUPS AND THE SUBQUOTIENT DECOMPOSITION

2.1. Parabolic subgroups. For a subset $J \subset S$, the *parabolic subgroup* $W_J = \langle s_i : s_i \in J \rangle$ is itself a Coxeter group (with the induced Coxeter matrix), and its length function is the restriction of that of W .

We fix a chain

$$W_0 = \{e\} \subset W_1 \subset W_2 \subset \cdots \subset W_n = W,$$

where $W_i = \langle s_1, \dots, s_i \rangle$ (using the *internal* numbering of generators described in Section 3). Since each W_i is parabolic, this is a chain of parabolic subgroups. The index

$$N_i = [W_i : W_{i-1}] = \frac{|W_i|}{|W_{i-1}|}$$

is the number of *parabolic subquotient* cosets at level i ; the group order factors as

$$|W| = N_1 \cdot N_2 \cdots N_n.$$

2.2. Minimal-length coset representatives.

Definition 2.1. For a parabolic subgroup $W_J \subset W$, the set of minimal-length left-coset representatives of $W_J \backslash W$ is

$${}^JW = \{w \in W : \ell(sw) > \ell(w) \text{ for all } s \in J\}.$$

That is, JW consists of elements with no left descent in J .

The fundamental theorem on parabolic cosets (see, e.g., Humphreys [2]) states:

Theorem 2.2. Every element $w \in W$ has a unique decomposition $w = u \cdot x$ with $u \in W_J$ and $x \in {}^JW$; moreover $\ell(w) = \ell(u) + \ell(x)$.

For the restricted quotient $W_{i-1} \backslash W_i$, set $D_i = S_{i-1} W_i$ (where $S_{i-1} = \{s_1, \dots, s_{i-1}\}$). These are the elements of W_i with no left descent in S_{i-1} , i.e., the minimal-length representatives of the left cosets $W_{i-1} \backslash W_i$. Clearly $|D_i| = N_i$.

2.3. The iterated factorization. Applying Theorem 2.2 repeatedly gives the core structural result:

Theorem 2.3 (Parabolic factorization). Every element $w \in W$ has a unique factorization

$$w = d_1 \cdot d_2 \cdots d_n$$

with $d_i \in D_i$ for each i . Moreover $\ell(w) = \sum_{i=1}^n \ell(d_i)$.

Proof. We argue by downward induction, extracting the rightmost factor first. Since $w \in W = W_n$, Theorem 2.2 applied to $W_{n-1} \subset W_n$ gives a unique decomposition $w = u_{n-1} \cdot d_n$ with $u_{n-1} \in W_{n-1}$ and $d_n \in D_n$. Apply the same theorem to $u_{n-1} \in W_{n-1}$ and $W_{n-2} \subset W_{n-1}$ to get $u_{n-1} = u_{n-2} \cdot d_{n-1}$, and continue. At the last step, $u_1 \in W_1 = \langle s_1 \rangle$ decomposes as $u_1 = d_1$ (since $D_1 = W_1$). Uniqueness and the length equality follow from Theorem 2.2 at each step. $\square$

Example 2.4 ($W(A_2) = S_3$). Label the generators as in Bourbaki: $s_1 = (12)$, $s_2 = (23)$. Then $W_1 = \{e, s_1\}$ and $W_2 = S_3$.

- $D_1 = W_1 = \{e, s_1\}$, so $N_1 = 2$.
- $D_2 = \{w \in W : \ell(s_1 w) > \ell(w)\} = \{e, s_2, s_2 s_1\}$, so $N_2 = 3$.

The six elements and their factorizations are:

w	d_1	d_2
e	e	e
s_1	s_1	e
s_2	e	s_2
$s_1 s_2$	s_1	s_2
$s_2 s_1$	e	$s_2 s_1$
$s_1 s_2 s_1$	s_1	$s_2 s_1$

Piece 1 (value 0 or 1, encoded as unsigned char 0 or 1) selects the $W_0 \setminus W_1$ coset; piece 2 (value 0, 1, or 2) selects the $W_1 \setminus W_2$ coset.

3. GENERATOR ORDERING

Atlas represents each d_i by its index $a_i \in \{0, 1, \dots, N_i - 1\}$ as an **unsigned char** (8-bit integer, values 0–254; value 255 is reserved as a sentinel). For this to work we need $N_i \leq 255$ for every level i .

With the standard *Bourbaki ordering* of generators, some classical types fail this constraint. For type B_n with generators $s_1 - s_2 - \dots - s_{n-1} \Rightarrow s_n$ (double bond at the right):

$$W_i = \langle s_1, \dots, s_i \rangle \cong W(A_i), \quad i < n, \quad W_n = W(B_n).$$

The final subquotient has size $N_n = |W(B_n)|/|W(A_{n-1})| = 2^n$, which overflows a byte for $n \geq 8$.

The fix is to *reverse* the generator order for types B , C , and D . Define the internal generators $t_j = s_{n+1-j}$ (so $t_1 = s_n$, the “special” end of the diagram). Then

$$W_r^{\text{int}} = \langle t_1, \dots, t_r \rangle \cong W(B_r), \quad N_r = \frac{|W(B_r)|}{|W(B_{r-1})|} = 2r.$$

The maximum subquotient size is $N_n = 2n$, which is ≤ 255 for $n \leq 127$.

Similarly, for type A_n (standard ordering) the subquotient at level r has size $N_r = r + 1$, so the maximum is $n + 1 \leq 255$ for $n \leq 254$.

Remark 3.1. *Atlas actually allows ranks up to `RANK_MAX`, and the comment in the source notes: “the unsigned char type miraculously suffices for all subquotients of all groups up to rank 128 (indeed, the biggest subquotient for B_{128} is of order 256), provided the generators are enumerated in an appropriate order.” The reversal of B , C , D is precisely this appropriate ordering. For the exceptional types, the sizes are small enough that no reordering is needed.*

Because of this reordering, the `WeylGroup` object carries two permutations, `d_in` and `d_out`, that translate between the *external* Bourbaki numbering (used in all user-facing output) and the *internal* numbering (used inside the transducer tables).

4. THE `WEYLELT` DATA TYPE

Definition 4.1. *A `WeylElt` is a fixed-length array*

$$(a_1, a_2, \dots, a_n) \in \prod_{i=1}^n \{0, 1, \dots, N_i - 1\},$$

stored as `pieces[0], \dots, pieces[n-1]` of type `unsigned char`. Piece i encodes which element of D_{i+1} appears in the factorization $w = d_1 d_2 \cdots d_n$.

Since the pieces are fixed-size and small, a `WeylElt` occupies exactly `RANK_MAX` bytes (currently 16 bytes), regardless of the group. This is the same size as a 128-bit integer, so comparison and copying are extremely cheap.

The identity element is the all-zeros tuple $(0, 0, \dots, 0)$. The longest element $w_0 \in W$ has $a_i = N_i - 1$ for every i (the maximal piece value at each level).

5. THE TRANSDUCER AUTOMATON

5.1. Overview. The multiplication $w \mapsto w \cdot s$ (right multiplication by a simple reflection) is computed using a system of precomputed finite-state automata called *transducers*, one per level. The Transducer at level r encodes the right-multiplication action of all generators $s_1, \dots, s_r$ on the coset representatives D_r .

5.2. Transitions and transductions. The key algebraic fact is: for a minimal-length coset representative $x \in D_r$ and a generator s_j with $j \leq r$, exactly one of the following holds:

- (1) **Transition:** $x \cdot s_j = x'$ for some other $x' \in D_r$ (with $x' \neq x$).
- (2) **Transduction:** $x \cdot s_j = s_k \cdot x$ for some generator s_k with $k < r$, i.e., x is fixed and the generator s_k must be “passed” to the lower-level transducers.

In case (1), the level- r piece changes; the multiplication stops at level r . In case (2), the level- r piece is unchanged; the emitted generator s_k is fed into the transducer at level $k' \leq r - 1$ (the highest level in the same Dynkin diagram component as k), and the process continues.

The two cases are distinguished by inspecting the transition table: a *shift* entry gives the new piece index x' , while a *transduction* entry gives the emitted generator s_k . In practice both are stored in a single table (indexed by current piece and incoming generator), using the convention that entries $< |D_r|$ are shifts and entries $\geq |D_r|$ encode transductions.

5.3. Right multiplication algorithm. Given $w = (a_1, \dots, a_n)$ and an (internally numbered) generator s , the algorithm to compute $w \cdot s$ proceeds as follows:

- (1) Let R be the index of the last generator in the Dynkin diagram component containing s (field `upper[s]` in Atlas). Set $s_{\text{cur}} = s$ and $i = R$.
- (2) Consult the Transducer at level i with current piece a_i and generator s_{cur} .
 - **Shift:** update $a_i \leftarrow a'_i$, stop. Return +1 if $a'_i > a_i$ (length increases), -1 if $a'_i < a_i$ (length decreases).
 - **Transduction:** emit s_k ; set $s_{\text{cur}} = s_k$ and $i \leftarrow i - 1$; go to step 2.

Note that the loop can only descend to lower indices, so it terminates in at most n steps. In practice the transduction chain is very short.

Remark 5.1. *Generators in different Dynkin diagram components always commute, so right multiplication by a generator s from one component never affects pieces belonging to a different component. The field `upper[s]` correctly restricts attention to the component of s .*

5.4. Constructing the transducer tables. The Transducer for level r is built at group construction time by a bootstrapping procedure, starting from the identity coset representative and iteratively discovering new representatives by applying the final generator s_r .

The procedure runs as follows. Initialize the automaton with a single state (the identity piece). For the first state and for each state added later:

- For all generators $t \leq r$: if the shift or transduction of the current state under t is not yet defined, create a new state xs (extending the normal form by s_r), and determine its table entries by analyzing the dihedral group orbit $\langle s, t \rangle$ acting on the current state.

The dihedral group $\langle s, t \rangle$ has order $2m$ where $m = m_{st}$ is the Coxeter matrix entry. There are three cases depending on the length of the orbit of the current element in the quotient:

- (1) Full orbit of size $2m$: the shift target for t is determined by traversing $m - 1$ upward steps from the bottom of the orbit.
- (2) Orbit of size m with a stationary step at the top: a transduction occurs; the emitted generator is the same as that emitted by the bottom of the orbit under the first generator.
- (3) Otherwise: the action of t takes the current state upward; nothing needs to be recorded (the table entry remains undefined until a future state defines it from the other direction).

The loop terminates because each iteration either defines a previously undefined table entry or creates a new state (and the number of states is bounded by $N_r \leq 255$).

6. ENUMERATION VIA MIXED-RADIX REPRESENTATION

6.1. The bijection. By Theorem 2.3, an element $w \in W$ is uniquely determined by the tuple $(a_1, \dots, a_n) \in \prod_{i=1}^n \{0, \dots, N_i - 1\}$. This gives a bijection

$$W \leftrightarrow \{0, 1, \dots, |W| - 1\}$$

via the *mixed-radix* (also called factorial-number system) formula

$$\phi(w) = a_1 + N_1(a_2 + N_2(a_3 + \dots + N_{n-1}a_n \dots)).$$

Piece 1 is the *least significant “digit”* and piece n is the most significant.

This bijection is implemented in Atlas by two methods:

- `to_big_int(w)`: computes $\phi(w)$ by evaluating the above expression using Horner’s method, starting from piece n and working down:

$$u \leftarrow 0, \quad u \leftarrow u \cdot N_i + a_i \quad (i = n, n - 1, \dots, 1).$$

The result is returned as a `big_int` to handle groups whose order exceeds 2^{64} .

- **toWeylElt(u)**: computes the inverse by successive Euclidean division:

$$a_i \leftarrow u \bmod N_i, \quad u \leftarrow \lfloor u/N_i \rfloor \quad (i = 1, 2, \dots, n).$$

6.2. Iteration over all elements. To iterate over all $|W|$ elements in order of their integer index, it suffices to iterate through all tuples $(a_1, a_2, \dots, a_n)$ with $0 \leq a_i < N_i$. In practice one can maintain a **WeylElt** and increment it lexicographically: increment a_1 and carry to a_2 when $a_1 = N_1$, etc. This gives the elements in an order consistent with the Bruhat order restricted to the chain $W_1, \dots, W_n$ (though not the full Bruhat order on W).

Example 6.1 (Continuing Example 2.4). *With $N_1 = 2$, $N_2 = 3$, the six tuples and their integer indices are:*

w	a_1	a_2	$\phi(w)$
e	0	0	0
s_1	1	0	1
s_2	0	1	2
$s_1 s_2$	1	1	3
$s_2 s_1$	0	2	4
$s_1 s_2 s_1$	1	2	5

7. DATA STRUCTURE SUMMARY

The complete picture is as follows.

WeylGroup object. Constructed once from the Cartan matrix. Contains:

- **d_transducer**: a vector of n **Transducer** objects. Transducer i stores a matrix **table** of size $N_{i+1} \times (i+2)$ (states $\times$ generators), whose entries encode transitions and transductions as described above.
- **d_in**, **d_out**: permutations translating between Bourbaki (external) and internal generator numbering.
- **upper**: for each (internal) generator s , the index of the last generator in the same Dynkin diagram component.
- **d_min_star**: for each generator s , the smallest-indexed generator that does not commute with s (or s itself if all others commute); used to speed up left multiplication.

WeylElt value. A fixed array of **RANK_MAX unsigned chars**. Piece i (0-indexed) holds $a_{i+1} \in \{0, \dots, N_{i+1} - 1\}$. Unused pieces (for generators beyond the rank) are held at zero. Operations:

- **Identity**: all pieces zero.

- **Right multiply by s :** `inner_mult`, complexity $O(n)$ in the worst case but typically $O(1)$ or $O(2)$ in practice.
- **Length:** sum of the lengths of the individual pieces (stored in each `Transducer`).
- **Comparison:** lexicographic order on pieces; consistent with grouping by Bruhat length but not with Bruhat order itself.
- **Word recovery:** read off the pieces from right to left, recovering the canonical reduced expression.

8. SIZE CONSTRAINTS AND RANK LIMITS

The constraint that each piece fits in an `unsigned char` (values 0–254, with 255 reserved) limits the achievable ranks:

Type	Internal generator order	Max level- r subquotient size
A_n	Bourbaki	$r + 1$
B_n, C_n	Reversed Bourbaki	$2r$
D_n	Reversed Bourbaki	$2r - 2$ (for $r \geq 2$)
E_6, E_7, E_8	Fixed (see Bourbaki)	≤ 240
F_4	Fixed	≤ 24
G_2	Fixed	≤ 6

In each case the bound is comfortably below 255. The combined constraint for a level- r transducer is actually $N_r + r \leq 256$: the N_r shift values and (at most) r possible transduction outputs must fit in a single byte. For C_{86} this gives $171 + 85 = 256$, just at the practical limit.

9. CONNECTION TO THE BRUHAT ORDER AND KAZHDAN–LUSZTIG THEORY

While the parabolic factorization is designed for efficient computation rather than reflection of Bruhat order, it does have representation-theoretic significance. The factorization $w = d_1 \cdots d_n$ is *length-additive*: $\ell(w) = \sum_i \ell(d_i)$. This means that a reduced word for w is obtained by concatenating reduced words for $d_1, \dots, d_n$ in order.

The parabolic subquotients also arise naturally when one stratifies the flag variety G/B by the Schubert cells corresponding to the chain $B \subset P_1 \subset P_2 \subset \cdots \subset G$ (where P_i is the parabolic subgroup whose Levi factor has Weyl group W_i).

In Atlas, the `WeylElt` type is also used to represent *twisted involutions* (elements $\sigma \in W$ satisfying $\sigma = \delta(\sigma^{-1})$ for a diagram involution δ), which parametrize the K -orbits on the flag variety needed for the computation of Kazhdan–Lusztig–Vogan polynomials.

REFERENCES

- [1] Fokko du Cloux, *A transducer approach to Coxeter groups*, Journal of Symbolic Computation **27** (1999), 311–324.
- [2] James E. Humphreys, *Reflection Groups and Coxeter Groups*, Cambridge Studies in Advanced Mathematics, vol. 29, Cambridge University Press, 1990.
- [3] Anders Björner and Francesco Brenti, *Combinatorics of Coxeter Groups*, Graduate Texts in Mathematics, vol. 231, Springer, 2005.
- [4] Jeffrey Adams, Marc van Leeuwen, Peter Trapa, and David A. Vogan Jr., *Unitary representations of real reductive groups*, preprint, [arXiv:1212.2192](#).